



Rust

Let's Get Rusty! -- Rust 入门指北: 基础设施

手把手 setup your develop env

1. 开发环境

Rust 当前的开发工具链，由 rustup + cargo 组成：

- rustup 负责开发环境的快速搭建&切换，相关组件源码一键下载
- cargo 包管理，编译&运行

1.1 rustup

rustup 工具，主要负责 rust toolchains，官方推荐的安装 setup 工具。

点击网站，获取下载命令：rustup.rs - The Rust toolchain installer

```
1 // 1. 基于 rust-init.sh 一键下载安装， 建议!!!
2 curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
3
4 // 2. ubuntu apt, 可能不是最新的版本,
5 sudo apt install rustup
6
7 // 3. macos brew
8 brew install rustup-init
9
```

```
10 // 4. build from source
11 https://github.com/rust-lang/rust
```

```
sphinx.shi@MBJ390Y7JFVR ~ % curl --proto 'https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
info: downloading installer
```

Welcome to Rust!

This will download and install the official compiler for the Rust programming language, and its package manager, Cargo.

Rustup metadata and toolchains will be installed into the Rustup home directory, located at:

`/Users/sphinx.shi/.rustup`

This can be modified with the `RUSTUP_HOME` environment variable.

The Cargo home directory is located at:

`/Users/sphinx.shi/.cargo`

This can be modified with the `CARGO_HOME` environment variable.

The `cargo`, `rustc`, `rustup` and other commands will be added to Cargo's bin directory, located at:

`/Users/sphinx.shi/.cargo/bin`

This path will then be added to your `PATH` environment variable by modifying the profile files located at:

`/Users/sphinx.shi/.profile`
`/Users/sphinx.shi/.zshenv`

You can uninstall at any time with `rustup self uninstall` and these changes will be reverted.

Current installation options:

default host triple: **aarch64-apple-darwin**
default toolchain: **stable (default)**
profile: **default**
modify PATH variable: **yes**

1) Proceed with standard installation (default - just press enter)
2) Customize installation
3) Cancel installation
>

直接一路enter 即可 (注意: 最后可能还需要根据提示手动 source 下)

1.1.1 [建议] completions

给终端添加补全, MacOS 的 zsh 为例:

```
1 # 1. 默认zsh 没有这个配置文件, 需要手动创建
2 mkdir ~/.zfunc
3
```

```
4 # 2. 添加到 ~/.zshrc 中
5 fpath+= ~/.zfunc
6
7 # 3. rustup 生成补全配置
8 rustup completions zsh > ~/.zfunc/_rustup
9
10 # 4. 手动刷新一下
11 exec zsh
```

1.1.2 [建议] 国内源替换

A. Rustup

源的替换可以参考 [rustup | 镜像站使用帮助](#) | [清华大学开源软件镜像站](#) | [Tsinghua Open Source Mirror](#)

MacOS 用的是 zsh，可以直接添加到 `~/.zshrc` 中的:

| bash 的话，也是一样直接添加到 `~/.bashrc`

```
1 sphinx.shi@MBJ390Y7JFVR ~ % cat .zshrc
2 export RUSTUP_UPDATE_ROOT=https://mirrors.tuna.tsinghua.edu.cn/rustup/rustup
3 export RUSTUP_DIST_SERVER=https://mirrors.tuna.tsinghua.edu.cn/rustup
```

1.2 cargo

cargo 无需额外安装，在完成上一步之后，就可以直接使用了

1.2.1 cargo 换源

参考: [crates.io-index.git](#) | [镜像站使用帮助](#) | [清华大学开源软件镜像站](#) | [Tsinghua Open Source Mirror](#)

Macos zsh 可以如下配置:

```
1 cat << EOF | tee -a ${CARGO_HOME:-$HOME/.cargo}/config.toml
2 [source.crates-io]
3 replace-with = 'mirror'
4
5 [source.mirror]
6 registry = "sparse+https://mirrors.tuna.tsinghua.edu.cn/crates.io-index/"
7 EOF
```

| `sparse+` 表示在使用稀疏索引;

cargo 1.68 版本开始支持稀疏索引：不再需要完整克隆 crates.io-index 仓库，可以加快获取包的速度

1.2.2 crates

crate 是 cargo 中对于第三方包的抽象，每一个三方库都是一个 crate，由于 cargo 统一运输，在编译的时候进行下载

可以类比 conan 中 recipe 的概念

crates.io: Rust Package Registry 是 rust 官方 package 管理中心，可以在这里找到实现的开源库

crates.io Browse All Crates | Sphinx

The Rust community's crate registry

Type 'S' or '/' to search

[Install Cargo](#) [Getting Started](#)

Instantly publish your crates and install them. Use the API to interact and find out more information about available crates. Become a contributor and enhance the site with your work.

96,049,917,620 Downloads

163,548 Crates in stock

New Crates	Most Downloaded	Just Updated
ldd_md_parse v0.1.0	syn	lox_utils v0.2.0
diff-man v0.1.0	bitflags	lox-library v0.2.0
reforge v0.1.0	proc-macro2	haveibeenpwned-downloader v0.3.0
black-fuchsia	quote	sequoia-sq

举个例子：

 crates.io

openssl

Sphinx

 Browse All Crates |  Sphinx ▼

Search Results for 'openssl'

Displaying 1-10 of 1517 total results

Sort by  Relevance ▼

openssl v0.10.68 

OpenSSL bindings

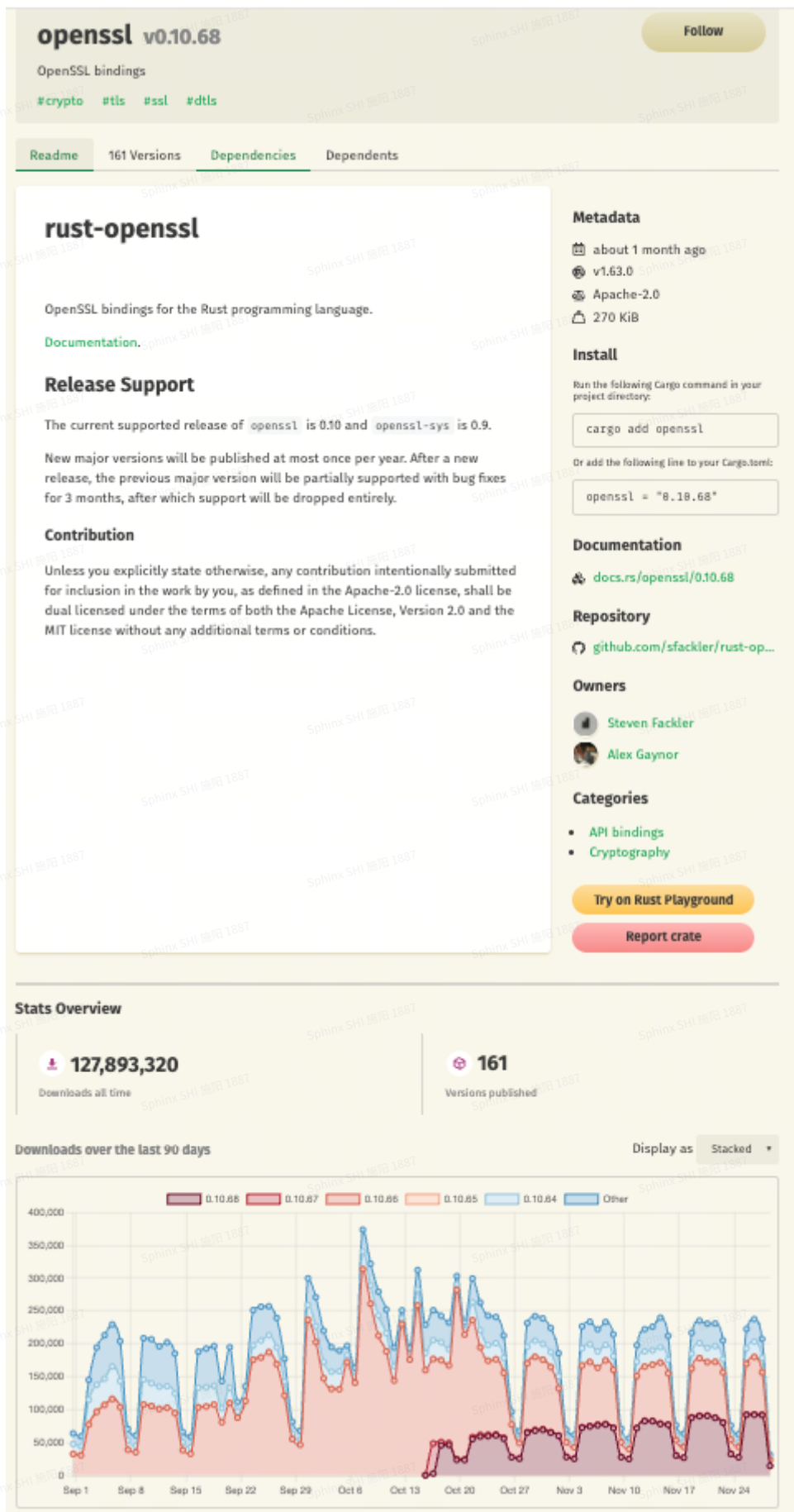
Repository

↓ All-Time: 127,893,320

↓ Recent: 17,180,575

🔄 Updated: about 1 month ago

点击进去就是详细的说明，可以安装说明进行安装使用；



1.3 Hello World!

完成安装之后，就可以开始coding了！

1.3.1 Setup for IDE

Rust 当前比较主流的IDE 有

- VS Code [Visual Studio Code - Code Editing. Redefined](#)
- RustRover [RustRover: Rust IDE by JetBrains](#)

⚠ RustRover 商用需付费！个人开发&教育用户免费

使用RustRover 的话，可以开箱即用，无需过多说明；

使用VS Code 的话，建议安装如下插件：

- [rust-analyzer - Visual Studio Marketplace](#) 语法高亮，提示补全，基于lsp 技术；
- [Cargo - Visual Studio Marketplace](#) 集成 cargo 及相关支持

一些其他可以根据个人需要定制, 如

- [Rust Doc Viewer - Visual Studio Marketplace](#)
- [Even Better TOML - Visual Studio Marketplace](#)

🔔 Cargo 使用toml 作为配置文件，所以建议安装toml 插件

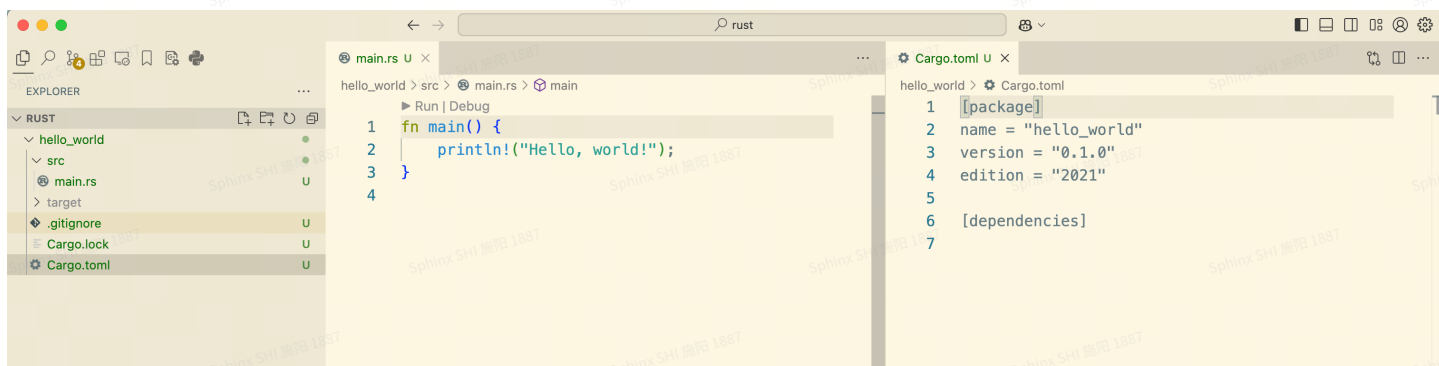
1.3.2 第一个工程！

```
1 phinx.shi@MBJ390Y7JFVR rust % cargo new hello_world --bin # 1
2     Creating binary (application) `hello_world` package
3 note: see more `Cargo.toml` keys and their definitions at https://doc.rust-
4     lang.org/cargo/reference/manifest.html
5 sphinx.shi@MBJ390Y7JFVR rust % code . # 2
```

#1 使用cargo 创建 hello_world 工程，`--bin` 显式指定工程是二进制的执行程序；

#2 `code .` 使用vscode 打开

没有问题后，将可以看到：



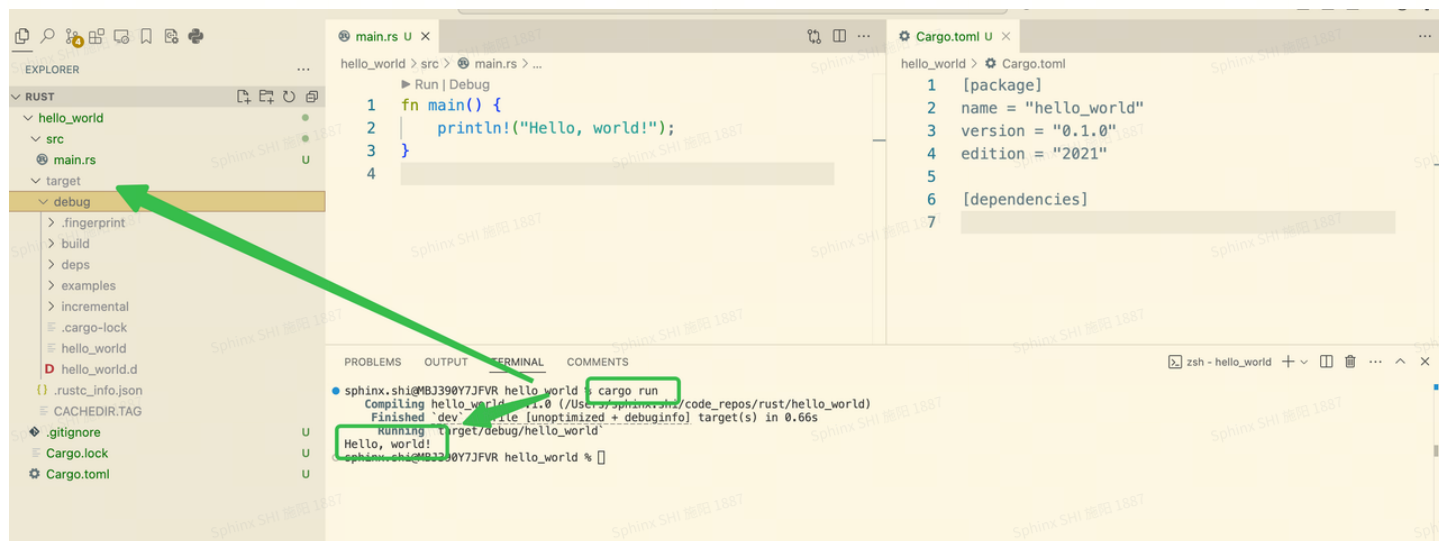
如截图就是 `cargo new` 生成的工程，主要文件就是

- main.rs
- Cargo.toml

打开终端运行 `cargo run`

`cargo run` 将直接进行编译+运行

也可以分开进行, `cargo build` , `cargo run` ;



在终端, 可以直接看到输出了 `Hello, world!` , 同时也可以看到cargo 创建的target 目录下的一些产物.

1.3.3 添加crate !

以floem 为例 <https://github.com/lapce/floem>

1. 添加库 `cargo add floem` , 将自动添加该库到Cargo.toml中



2. Coding

安装官方的api 添加如下代码:

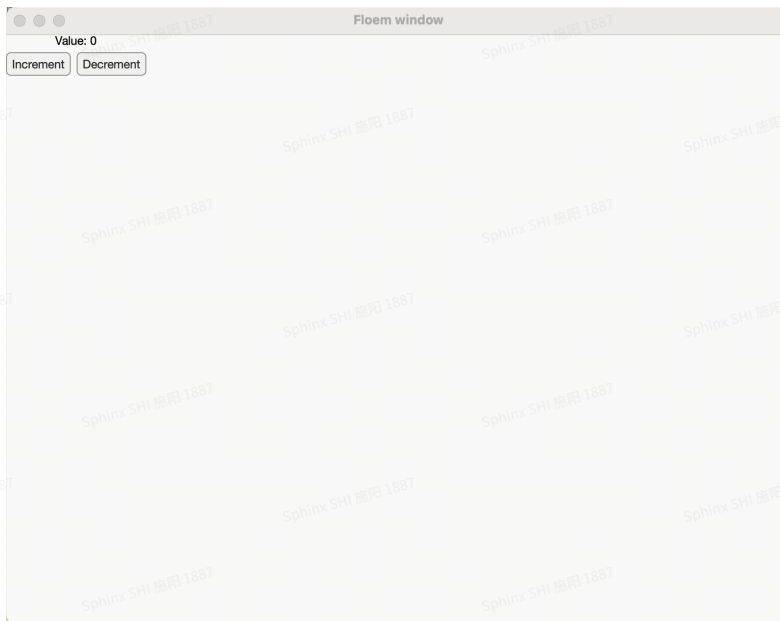
```
1 use floem::{
2     prelude::create_rw_signal,
3     views::{button, dyn_view, Decorators},
4     IntoView,
5 };
6
7 fn app_view() -> impl IntoView {
8     let mut counter = create_rw_signal(0);
9     (
10         dyn_view(move || format!("Value: {}", counter)),
11         (
12             button("Increment").action(move || counter += 1),
13             button("Decrement").action(move || counter -= 1),
14         )
15     ).style(|s| s.flex_row().gap(6)),
16 )
17     .style(|s| s.flex_col().gap(6).items_center())
18 }
19
20 fn main() {
21     floem::launch(app_view);
22 }
```

3. Building & Runing

完成编码之后, 使用cargo 命令:

```
1 cargo run
```

4. 运行结果



2. Rustup & Cargo 介绍

Copilot 总结

 **GitHub Copilot**

> Used 1 reference

rustup 是一个用于管理 Rust 工具链的命令行工具。它允许你安装和管理多个版本的 Rust 编译器和相关工具。通过 **rustup**，你可以轻松地切换不同的 Rust 版本，安装特定的工具链（如稳定版、测试版和夜间版），并保持你的工具链最新。

一些常用的 **rustup** 命令包括：

- **rustup install <version>**：安装指定版本的 Rust。
- **rustup update**：更新 Rust 工具链到最新版本。
- **rustup default <version>**：设置默认的 Rust 版本。
- **rustup toolchain list**：列出已安装的工具链。

例如，安装最新的稳定版 Rust：

```
rustup install stable
```

设置默认工具链为稳定版：

```
rustup default stable
```

2.1 Rustup 入门指北

- Channel
- Toolchain
- Component
- Profile
- Proxy

2.1.1 Channel

Rust 有三个不同的发布渠道：

- 稳定版 (stable)
- 测试版 (beta)
- 夜间版 (nightly)

区别就是，稳定版每六周发布一次，测试版是下一个稳定版的预览，夜间版每天发布。

rustup 工具可以帮助安装、更新和切换不同的渠道。

通过 `rustup toolchain` 来查看对应的提示信息

```
1  % rustup toolchain
2  Modify or query the installed toolchains
3
4  Usage: rustup toolchain <COMMAND>
5
6  Commands:
7  list      List installed toolchains
8  install   Install or update a given toolchain
9  uninstall Uninstall a toolchain
10 link      Create a custom toolchain by symlinking to a directory
11 help      Print this message or the help of the given subcommand(s)
12
13 Options:
14 -h, --help  Print help
15
16 Discussion:
17     Many `rustup` commands deal with *toolchains*, a single
18     installation of the Rust compiler. `rustup` supports multiple
19     types of toolchains. The most basic track the official release
20     channels: 'stable', 'beta' and 'nightly'; but `rustup` can also
21     install specific toolchains from the official archives, toolchains for
22     alternate host platforms, and from local builds ('custom toolchains').
23
24     Standard release channel toolchain names have the following form:
25
26     <channel>[-<date>][-<host>]
27
28     <channel> = stable|beta|nightly|<major.minor>|
29     <major.minor.patch>
30
31     <date> = YYYY-MM-DD
32     <host> = <target-triple>
```

```

32 'channel' is a named release channel, a major and minor version
33 number such as `1.42`, or a fully specified version number, such
34 as `1.42.0`. Channel names can be optionally appended with an
35 archive date, as in `nightly-2014-12-18`, in which case the
36 toolchain is downloaded from the archive for that date.
37
38 The host may be specified as a target triple. This is most useful
39 for installing a 32-bit compiler on a 64-bit platform, or for
40 installing the [MSVC-based toolchain] on Windows. For example:
41
42     $ rustup toolchain install stable-x86_64-pc-windows-msvc
43
44 For convenience, omitted elements of the target triple will be
45 inferred, so the above could be written:
46
47     $ rustup toolchain install stable-msvc
48
49 The `rustup default` command may be used to both install and set
50 the desired toolchain as default in a single command:
51
52     $ rustup default stable-msvc
53
54 rustup can also manage symlinked local toolchain builds, which are
55 often used for developing Rust itself. For more information see
56 `rustup toolchain help link`.

```

2.1.2 Toolchains

在确认需要使用的channel 之后，就需要确认toolchains 了，通常channel 都是stable的，然后 toolchains 设置当前的运行平台，比如：

```

sphinx.shi@MBJ390Y7JFVR rust % rustup toolchain list
stable-aarch64-apple-darwin (default)

```

可以理解为，channel - toolchains ，就决定了实际的编译工具链的情况，可以理解为gcc/clang 不同的编译器版本设置，它的命名方式如下：

```

1 <channel>[-<date>][-<host>]
2
3 <channel>           = stable|beta|nightly|<major.minor>|<major.minor.patch>
4 <date>              = YYYY-MM-DD
5 <host>              = <target-triple>
6

```

也支持自定义自己想要的，使用如下这个命令：

```
1 $ rustup toolchain link my-toolchain path/to/my/toolchain/sysroot
```

实际，作为应用的开发，应该很少使用到定制的工具链，这个应该是在我们使用一些定制的工具链时才需要进行手动指定的；

想到的一个场景是：某个芯片厂商根据自己的芯片进行了rustc及相关的定制，在进行底层相关的coding时，要使用其特性，这个时候就需要进行手动link切换了，也可以写到配置文件中（~/rustup/settings.toml）

2.1.3 Components

就是toolchains下可使用的组件，可以使用 `rustup component list` 查看。

一般都是这些：

- rustc：Rust 编译器和 Rustdoc。
- cargo：包管理器和构建工具。
- rustfmt：自动格式化代码的工具。
- rust-std：Rust 标准库。
- rust-docs：本地 Rust 文档副本。
- rust-analyzer：提供编辑器和 IDE 支持的语言服务器。
- clippy：提供额外检查的 lint 工具。
- miri：实验性的 Rust 解释器。
- rust-src：Rust 标准库的本地源代码副本。
- rust-mingw：用于 x86_64-pc-windows-gnu 平台的链接器和平台库。
- llvm-tools：LLVM 工具集合。
- rustc-dev：包含编译器作为库。

使用 `rustup component add` 添加你需要的组件，比如想看标准库的源码，就可以使用

```
rustup component add rust-src 添加。
```

详细的完整信息，可以从这里看到：<https://rust-lang.github.io/rustup-components-history/>

2.1.4 Profile

rustup 的“配置文件”，它们是一些组件的分组，可以在安装新的 Rust 工具链时选择下载。

目前可用的配置文件有 minimal、default 和 complete：

- **minimal** 配置文件包含尽可能少的组件以获得一个可工作的编译器（rustc、rust-std 和 cargo）。建议在不使用本地文档的 Windows 系统和 CI 中使用此配置文件（大量文件可能会导致某些杀毒软件的问题）。
- **default** 配置文件包含 minimal 配置文件中的所有组件，并添加了 rust-docs、rustfmt 和 clippy。rustup 默认使用此配置文件，推荐用于一般用途。
- **complete** 配置文件包含通过 rustup 可用的所有组件。此配置文件不应使用，因为它包含元数据中曾经包含的每个组件，因此几乎总是会失败。如果需要安装开发工具（如 miri 或 IDE 集成工具 rust-analyzer），应使用 default 配置文件并手动安装所需的附加组件。

要更改 rustup 默认使用的配置文件，可以使用 `rustup set profile` 命令。例如，选择 minimal 配置文件：

```
rustup set profile minimal
```

也可以在安装工具链时直接选择使用的配置文件：

```
rustup install --profile <name>
```

首次安装 rustup 时，也可以通过交互选择“自定义安装”选项或通过传递 `--profile=<name>` 标志来选择默认配置文件。配置文件只会影响新安装的工具链，之后仍然可以使用 `rustup component add` 安装单个组件。

2.1.5 Proxies

rustup 提供了一些常见 Rust 工具的包装器，称为代理（proxies），这些代理代表由各种组件提供的命令。

目前 rustup 中的代理列表如下：

- **rustc**：Rust 编程语言的编译器，由 rustc 组件提供。
- **rustdoc**：由 rustc 组件分发的工具，帮助生成 Rust 项目的文档。
- **cargo**：Rust 包管理器，下载依赖项、编译包、创建可分发包并上传到 crates.io，由 cargo 组件提供。
- **rust-lldb**、**rust-gdb** 和 **rust-gdbgui**：分别是 lldb、gdb 和 gdbgui 调试器的简单包装器，通过脚本接口添加 Rust 值的美化打印和一些便利功能。
- **rust-analyzer**：Rust IDE 集成工具的一部分，实现语言服务器协议，允许 IDE 和编辑器（如 Visual Studio Code、Vim 或 Emacs）访问编辑的 Rust 代码的语义，由 rust-analyzer 组件提供。
- **cargo-clippy** 和 **clippy-driver**：与 clippy lint 工具相关，提供常见错误和风格选择的额外检查，由 clippy 组件提供。
- **cargo-miri**：Rust 中级中间表示（MIR）的实验性解释器，由 miri 组件提供。
- **rls**：已被 rust-analyzer 取代的 IDE 工具，由 rls 组件提供。

2.2 基本使用

- Update

- 更新 stable-toolchains `rustup update`
- 更新 rustup self `rustup self update`

- Overried

- 在工程目录下，设置指定的 toolchains
 - `rustup override set nightly-xxxx-xx-xx`
 - `rust-toolchain.toml`

```
1 [toolchain]
2 channel = "aarch64-apple-darwin"
3 components = [ "rustfmt", "rustc-dev" ]
4 targets = [ "wasm32-unknown-unknown", "thumbv2-none-eabi" ]
5 profile = "minimal"
```

- Cross-compilation

Rust 支持许多平台, 为许多平台发布了标准库的二进制版本, 并为一些平台发布了完整的编译器。

首次安装工具链时, rustup 仅安装主机平台 (当前运行的架构和操作系统) 的标准库。要编译到其他平台, 必须安装其他目标平台。这可以通过 `rustup target add` 命令完成。

例如, 添加 Android 目标:

```
$ rustup target add arm-linux-androideabi
```

安装 arm-linux-androideabi 目标后, 可以通过传递 `--target` 标志使用 Cargo 为 Android 构建:

```
$ cargo build --target=arm-linux-androideabi
```

注意, `rustup target add` 仅为给定目标安装 Rust 标准库。通常还需要其他工具, 特别是链接器。

例如, 要交叉编译到 Android, 需要安装 Android NDK。

要为非默认工具链安装目标, 请使用 `--toolchain` 参数:

```
$ rustup target add --toolchain <toolchain> <target>...
```

查看可用目标列表:

```
$ rustup target list
```

移除已添加的目标:

```
$ rustup target remove <target>
```

2.2.1 环境变量

2.2.1.1 RUSTUP_HOME

默认: `~/.rustup` 或 `%USERPROFILE%/.rustup`

设置 rustup 根文件夹, 用于存储已安装的工具链和配置选项。

2.2.1.2 RUSTUP_TOOLCHAIN

默认: 无

如果设置, 将覆盖所有 Rust 工具调用使用的工具链。应安装具有此名称的工具链, 否则调用将失败。可以指定自定义工具链、可安装工具链或工具链的绝对路径。

2.2.1.3 RUSTUP_DIST_SERVER

默认: `https://static.rust-lang.org`

设置下载与 Rust 相关的静态资源的根 URL。可以更改为使用本地镜像或测试暂存目录中的二进制文件。

2.2.1.4 RUSTUP_DIST_ROOT

默认: `https://static.rust-lang.org/dist`

已弃用默认, 请使用 `RUSTUP_DIST_SERVER`。

2.2.1.5 RUSTUP_UPDATE_ROOT (

默认: `https://static.rust-lang.org/rustup`

设置下载自我更新的根 URL

2.2.1.6 RUSTUP_IO_THREADS

不稳定, 默认: 报告的 CPU 数量

设置执行关闭 IO 的线程数。设置为 1 强制单线程 IO 进行故障排除, 或设置为任意数目以覆盖自动检测。

2.2.1.7 RUSTUP_TRACE_DIR

不稳定, 默认: 无跟踪

启用跟踪并确定将写入跟踪的目录。跟踪形式为 PID.trace。可以通过 Catapult 项目跟踪查看器读取跟踪。

2.2.1.8 RUSTUP_DEBUG

不稳定

设置时, 启用 rustup 的调试日志记录。

2.2.1.9 RUSTUP_TERM_COLOR

默认：自动

控制终端中是否使用彩色输出。设置为 auto 仅在 tty 流中使用颜色，设置为 always 始终启用颜色，设置为 never 禁用颜色。

2.2.1.10 RUSTUP_UNPACK_RAM

不稳定，默认

可用内存或无法确定时为 500MiB，最小 210MiB）：限制 rustup 在解压时用于 IO 任务的 RAM 数量。

2.2.1.11 RUSTUP_NO_BACKTRACE

禁用非 panic 错误的回溯，即使设置了 RUST_BACKTRACE。

2.2.1.12 RUSTUP_PERMIT_COPY_RENAME

不稳定

设置时，允许 rustup 在重命名尝试导致跨设备链接错误时回退到复制文件。这些错误发生在 Docker 使用的 OverlayFS 上。此功能牺牲了一些事务保护，可能随时被移除。仅限 Linux。

2.3 Cargo 入门指北

GitHub Copilot

Cargo 是 Rust 的包管理器和构建系统。它负责管理 Rust 项目的依赖、编译项目、创建可分发包并将其上传到 crates.io（Rust 社区的包注册表）。以下是 Cargo 的一些主要功能和命令：

主要功能

1. 依赖管理：Cargo 可以自动下载和管理项目所需的依赖库。
2. 构建系统：Cargo 提供了简单的命令来编译项目。
3. 测试：Cargo 支持运行单元测试和集成测试。
4. 发布：Cargo 可以将包发布到 crates.io。

Cargo 主要做了下面这四件事：

- 引入两个元数据文件，包含各种包的信息
- 获取并构建软件包的依赖项
- 调用 rustc 或其他具有正确参数的构建工具来构建你的软件包
- 引入惯例，使 Rust 包的工作更容易

在很大程度上，Cargo 将构建一个特定程序或库所需的命令规范化，实现同一个命令可以用来构建不同的工件，他们的实现方式，依赖多少库，项目结构如何。就是，一旦你知道如何构建一个基于 Cargo 的项目，那就知道如何构建所有的项目。

2.3.1 基本使用

2.3.1.1 cargo new

```

1 cargo new --help
2 Create a new cargo package at <path>
3
4 Usage: cargo new [OPTIONS] <PATH>
5
6 Arguments:
7     <PATH>
8
9 Options:
10     --vcs <VCS>          Initialize a new repository for the given
                             version control system, overriding a global configuration. [possible values:
                             git, hg, pijul, fossil, none]
11     --bin                 Use a binary (application) template [default]
12     --lib                 Use a library template
13     --edition <YEAR>     Edition to set for the crate generated [possible
                             values: 2015, 2018, 2021, 2024]
14     --name <NAME>        Set the resulting package name, defaults to the
                             directory name
15     --registry <REGISTRY> Registry to use
16     -v, --verbose...      Use verbose output (-vv very verbose/build.rs
                             output)
17     -q, --quiet           Do not print cargo log messages
18     --color <WHEN>       Coloring: auto, always, never
19     --config <KEY=VALUE> Override a configuration value
20     -Z <FLAG>            Unstable (nightly-only) flags to Cargo, see
                             'cargo -Z help' for details
21     -h, --help           Print help
22
23 Manifest Options:
24     --locked             Assert that `Cargo.lock` will remain unchanged
25     --offline            Run without accessing the network
26     --frozen             Equivalent to specifying both --locked and --offline
27
28 Run `cargo help new` for more detailed information.

```

创建新的bin or lib 库，通过options 来指定自己需要的类型；

基本的项目结构

```

1 % tree ./ -L 3
2 ./
3 └─ hello_world
4     │   └─ Cargo.lock
5     │   └─ Cargo.toml
6     │   └─ src
7     │       └─ main.rs

```

```
8      └─ target
9      │   └─ CACHEDIR.TAG
10     └─ debug
11
12 5 directories, 4 files
```

Cargo.toml 和 src/main.rs 两个最基础的文件：

- Cargo.toml

- 项目信息

```
1 cat Cargo.toml
2 [package] # package 的基本信息
3 name = "hello_world"
4 version = "0.1.0"
5 edition = "2021"
6
7 [dependencies] # 依赖库的name-version
8 floem = "0.2.0"
```

- 依赖也支持git 协议下载，如

```
1 [package]
2 name = "hello_world"
3 version = "0.1.0"
4 [dependencies]
5 regex = { git = "https://github.com/rust-lang/regex.git" }
```

- main.rs 代码

- Cargo.lock

GitHub Copilot

Cargo.lock 是一个由 Cargo 生成的文件，用于锁定项目的依赖版本。它确保在不同的构建环境中使用相同的依赖版本，从而保证构建的一致性。以下是 **Cargo.lock** 的一些关键点：

作用

1. 版本锁定：记录项目及其所有依赖项的具体版本，确保在不同环境中构建时使用相同的版本。
2. 依赖树：包含项目的完整依赖树，包括直接依赖和间接依赖。

工作原理

- 当你第一次运行 **cargo build** 或 **cargo update** 时，Cargo 会解析 **Cargo.toml** 中指定的依赖，并生成 **Cargo.lock** 文件。
- 如果 **Cargo.lock** 已存在，Cargo 会使用其中指定的版本，而不是重新解析 **Cargo.toml** 中的版本范围。
- 如果你手动修改了 **Cargo.toml** 中的依赖版本，Cargo 会更新 **Cargo.lock** 文件以反映这些更改。

2.3.1.2 cargo update

- 更新当前的所有dependencise
- 如果只更新某一个, 指定一下就可以了 `cargo update crate-name`

如果只更新某一个, 指定一下就可以了`cargo update crate-name`

2.3.1.3 cargo test

- 运行源码中写的 ut 和documentation tests

2.3.1.4 cargo build

- 进行编译

2.3.1.5 cargo run

- 运行编译后的程序

2.3.1.6 cargo add/remove

- 添加/移除 相关的crate

2.3.1.7 CI

支持现有的使用yaml 配置的常用CI 设施, 比如:

- Circle CI
- Github CI
- Gitlab CI

2.3.2 常用的环境变量

2.3.2.1 Cargo 读取的环境变量

你可以覆盖这些环境变量来改变 Cargo 在系统上的行为:

- **CARGO_LOG**: 设置为 `trace`、`debug` 或 `warn` 以启用调试日志记录, 通常用于调试。
- **CARGO_HOME**: 覆盖 Cargo 本地缓存目录的位置, 默认在 `$HOME/.cargo` (Windows 上为 `%USERPROFILE%\.cargo`) 。
- **CARGO_TARGET_DIR**: 生成的所有工件的存放位置, 相对于当前工作目录。
- **CARGO**: 如果设置, Cargo 将使用此值而不是自动检测的路径来构建 crate 和执行构建脚本及外部子命令。
- **RUSTC**: 指定 Cargo 执行的编译器, 而不是默认的 `rustc` 。
- **RUSTC_WRAPPER**: 指定一个包装器来代替 `rustc`, 用于设置构建缓存工具如 `sccache` 。
- **RUSTC_WORKSPACE_WRAPPER**: 为工作区成员指定一个包装器, 类似于

- **RUSTDOC**: 指定 Cargo 执行的 `rustdoc` 实例。
- **RUSTDOCFLAGS**: 传递给所有 `rustdoc` 调用的自定义标志列表。
- **CARGO_ENCODED_RUSTDOCFLAGS**: 以 0x1f 分隔的自定义标志列表，传递给所有 `rustdoc` 调用。
- **RUSTFLAGS**: 传递给所有编译器调用的自定义标志列表。
- **CARGO_ENCODED_RUSTFLAGS**: 以 0x1f 分隔的自定义标志列表，传递给所有编译器调用。
- **CARGO_INCREMENTAL**: 设置为 1 强制启用增量编译，设置为 0 强制禁用。
- **CARGO_CACHE_RUSTC_INFO**: 设置为 0 禁用编译器版本信息缓存。
- **HTTPS_PROXY** 或 `https_proxy` 或 `http_proxy`: HTTP 代理设置。
- **HTTP_TIMEOUT**: HTTP 超时时间（秒）。
- **TERM**: 设置为 `dumb` 禁用进度条。
- **BROWSER**: 指定用于 `cargo doc --open` 的浏览器。
- **RUSTFMT**: 指定 Cargo 执行的 `rustfmt` 实例。

2.3.2.2 Cargo 设置的环境变量

Cargo 在编译时会向 crate 暴露这些环境变量：

- **CARGO**: 执行构建的 Cargo 二进制路径。
- **CARGO_MANIFEST_DIR**: 包含包清单的目录。
- **CARGO_PKG_VERSION**: 包的完整版本。
- **CARGO_PKG_VERSION_MAJOR**: 包的主版本号。
- **CARGO_PKG_VERSION_MINOR**: 包的次版本号。
- **CARGO_PKG_VERSION_PATCH**: 包的补丁版本号。
- **CARGO_PKG_VERSION_PRE**: 包的预发布版本。
- **CARGO_PKG_AUTHORS**: 包清单中的作者列表。
- **CARGO_PKG_NAME**: 包的名称。
- **CARGO_PKG_DESCRIPTION**: 包的描述。
- **CARGO_PKG_HOMEPAGE**: 包的主页。
- **CARGO_PKG_REPOSITORY**: 包的仓库地址。
- **CARGO_PKG_LICENSE**: 包的许可证。
- **CARGO_PKG_LICENSE_FILE**: 包的许可证文件。
- **CARGO_PKG_RUST_VERSION**: 包支持的最低 Rust 版本。

- **CARGO_PKG_README**: 包的 README 文件路径。
- **CARGO_CRATE_NAME**: 当前编译的 crate 名称。
- **CARGO_BIN_NAME**: 当前编译的二进制文件名称。
- **OUT_DIR**: 构建脚本输出的目录。
- **CARGO_BIN_EXE_<name>**: 二进制目标的绝对路径。
- **CARGO_PRIMARY_PACKAGE**: 如果正在构建的包是主要包, 则设置此变量。
- **CARGO_TARGET_TMPDIR**: 集成测试或基准测试代码的临时目录。
- **CARGO_RUSTC_CURRENT_DIR**: `rustc` 的调用路径 (仅限夜间版)。

2.3.2.3 动态库路径

Cargo 在编译和运行二进制文件时设置动态库路径, 帮助定位构建过程中使用的共享库。变量名称因平台而异:

- Windows: `PATH`
- macOS: `DYLD_FALLBACK_LIBRARY_PATH`
- Unix: `LD_LIBRARY_PATH`
- AIX: `LIBPATH`

2.3.2.4 构建脚本的环境变量

Cargo 在运行构建脚本时设置这些环境变量:

- **CARGO**: 执行构建的 Cargo 二进制路径。
- **CARGO_MANIFEST_DIR**: 包含包清单的目录。
- **CARGO_MANIFEST_LINKS**: 包清单中的链接值。
- **CARGO_MAKEFLAGS**: 包含并行化子进程所需的参数。
- **CARGO_FEATURE_<name>**: 为每个激活的特性设置的环境变量。
- **CARGO_CFG_<cfg>**: 为每个配置选项设置的环境变量。
- **OUT_DIR**: 输出和中间工件的目录。
- **TARGET**: 正在编译的目标三元组。
- **HOST**: Rust 编译器的主机三元组。
- **NUM_JOBS**: 并行作业数。
- **OPT_LEVEL**、**DEBUG**: 当前构建配置的优化级别和调试信息。
- **PROFILE**: 构建配置 (`release` 或 `debug`)。
- **DEP_<name>_<key>**: 链接的依赖项信息。

- **RUSTC、RUSTDOC**: Cargo 解析使用的编译器和文档生成器。
- **RUSTC_WRAPPER、RUSTC_WORKSPACE_WRAPPER**: Cargo 使用的 `rustc` 包装器。
- **RUSTC_LINKER**: Cargo 解析使用的链接器路径。
- **CARGO_ENCODED_RUSTFLAGS**: 传递给 `rustc` 的额外标志。

2.3.2.5 3rd 方子命令的环境变量

Cargo 向第三方子命令（如 `cargo-foobar`）暴露的环境变量：

- **CARGO**: 执行构建的 Cargo 二进制路径。
- **CARGO_MAKEFLAGS**: 包含并行化子进程所需的参数。

2.3.3 自定义编译脚本

 Sphinx SHI 施阳 TODO

2.3.4 Workspace

 Sphinx SHI 施阳 TODO

3. Edition 指北

Rust 中通过Edition 来明确&释放对应的语言支持的特性，类似c11，c23，cpp11，cpp14版本；三年一个版本迭代，第一个版本是2015 确定的，所以就是Rust 2015，后面版本也是如此定义的。具体的版本变化和迭代可以参考官方的edition book：

<https://doc.rust-lang.org/edition-guide/introduction.html>

3.1 Editions

3.1.2 Current List

- Rust 2015
- Rust 2018
- Rust 2021
- Rust 2024 not released yet

4. Refs

The rustup book: <https://rust-lang.github.io/rustup/index.html>

The cargo book: <https://doc.rust-lang.org/cargo/index.html>

The Edition Guide: <https://doc.rust-lang.org/edition-guide/introduction.html>